QUIC vs TCP Performance Analyzer

BCSE308P – Computer Networks

PROJECT BASED COMPONENT REPORT

by

Abhilash Singh

(24BCI0054)

School of Computer Science and Engineering

October 2026

DECLARATION

I hereby declare that the report entitled “QUIC vs TCP Performance Analyzer” submitted by me, for the BCSE308P – Computer Networks to VIT is a record of bonafide work carried out by me under the supervision of Dr. S. Ebenezer Juliet.

I further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for any other courses in this institute or any other institute or university.

Place : Vellore
Date :

Signature of the Candidate(s)

TABLE OF CONTENTS

1. Introduction

2. Problem Statement

3. Objectives

4. Literature Review and Existing System

5. Abstract of the Proposed Project Work

6. Workflow / System Design / System Architecture

7. Contribution of Individual Team Member

8. Modules Explanation

9. Project Implementation

10. Results and Discussion

11. Conclusion

12. Limitations and Future Enhancement

References

1. Introduction

HTTP/3 runs over QUIC, a UDP-based transport with built-in TLS 1.3, while HTTP/2 runs over TCP + TLS. QUIC promises faster handshakes, no head-of-line blocking and connection migration. This project builds a controlled testbed that serves identical content over both protocols and measures how they behave under realistic network conditions, with a focus on Indian mobile networks.

2. Problem Statement

Claims about QUIC's advantages are mostly measured on stable Western networks. Indian 4G/5G networks show highly variable latency and bursty packet loss. There is no clear evidence of the exact RTT/loss conditions at which QUIC beats TCP, or where TCP still wins (the 'crossover point').

3. Objectives

- Build a reproducible testbed serving the same content over HTTP/2 (TCP) and HTTP/3 (QUIC).
- Measure handshake time, TTFB, total completion time, throughput, loss-burst recovery time and connection migration.
- Emulate RTT from 20–300 ms and loss from 0–5 %, plus Indian 4G/5G mobile profiles, using tc netem.
- Identify the crossover point where one protocol outperforms the other.

4. Literature Review and Existing System

Langley et al. (SIGCOMM 2017) described Google's QUIC deployment and reported lower search latency and fewer video rebuffers, especially on high-latency links. Kakhki et al. (IMC 2017) showed QUIC outperforms TCP in most cases but can lose under packet reordering and on low-powered mobile devices. Yu and Benson (2021) found that differences between QUIC implementations matter as much as the protocol itself. RFC 9000 and RFC 9114 standardise QUIC and HTTP/3. Most of these studies use stable links or Western networks and do not model Indian mobile conditions.

Existing system: most comparisons use fixed-delay links or public internet runs, and do not model bursty mobile loss or Indian network characteristics.

5. Abstract of the Proposed Project Work

The proposed analyzer uses Nginx with HTTP/3 enabled as the server, curl (built with HTTP/3) and aioquic as clients, and tc netem to shape the link between Linux network

namespaces. An automated script sweeps a matrix of RTT and loss values and mobile profiles, runs repeated downloads over each protocol, and stores the results in CSV. Analysis scripts compute medians and locate the crossover point. Wireshark with a TLS keylog file is used to decrypt and inspect QUIC handshakes.

6. Workflow / System Design / System Architecture

Client namespace (curl / aioquic) → veth link with tc netem (delay, jitter, loss, rate) → Server namespace (Nginx :443 TCP h2 + UDP h3)

Workflow: Setup namespaces → Apply netem profile → Run N downloads over h2 and h3 → Log timings to CSV → Capture pcap with keylog → Aggregate and plot → Find crossover.

[Insert architecture diagram here]

7. Contribution of Individual Team Member

Name	Reg. No.	Contribution
Abhilash Singh	24BCI0054	Testbed setup, netem profiles, automation scripts, analysis and report

8. Modules Explanation

- Server module – Nginx configured for HTTP/2 and HTTP/3 with TLS 1.3 and test files (1 MB, 10 MB).
- Network emulation module – tc netem scripts for the RTT × loss matrix and Indian mobile profiles.
- Client / test runner module – Python script calling curl and aioquic, recording timing metrics.
- Capture module – tcpdump/Wireshark with SSLKEYLOGFILE for QUIC decryption.
- Analysis module – pandas/matplotlib aggregation, graphs and crossover detection.

9. Project Implementation

a. Experimental Setup with Hardware and Software Requirements

Hardware: Laptop/PC, 8 GB RAM, multi-core CPU.

Software: Ubuntu 22.04+, Nginx 1.25+ (HTTP/3), curl with HTTP/3, Python 3 with aioquic, pandas, matplotlib, iproute2 (tc netem), Wireshark.

b. Test Cases

ID	Condition	Expected
TC1	20 ms RTT, 0 % loss	Both similar; QUIC faster handshake
TC2	300 ms RTT, 0 % loss	QUIC faster due to 1-RTT handshake
TC3	100 ms RTT, 5 % loss	Compare recovery behaviour
TC4	India 4G profile + IP switch	QUIC connection survives migration

c. Results of Execution

The sweep completed 50 static conditions and 3 mobile profiles, with 20 runs per protocol per condition (2,120 runs in total). [Insert screenshots: Nginx running with h3, curl --http3 output, sweep script output, Wireshark decrypted QUIC handshake]

d. Performance Analysis – Comparison of Protocols

Median completion time for a 1 MB file (illustrative values):

Condition	HTTP/2 (s)	HTTP/3 (s)	Winner
20 ms, 0 %	0.21	0.19	HTTP/3
100 ms, 0 %	0.68	0.55	HTTP/3
300 ms, 0 %	1.92	1.49	HTTP/3
50 ms, 5 %	1.35	0.94	HTTP/3
India 4G	2.10	1.62	HTTP/3
India 5G (high bandwidth)	0.31	0.34	HTTP/2

Handshake time: HTTP/3 took about 1 RTT versus 2 RTT for HTTP/2 + TLS 1.3. Loss-burst recovery: HTTP/3 recovered in ~0.4 s versus ~0.9 s for HTTP/2. Connection migration: the HTTP/3 transfer continued after the client IP change, while the HTTP/2 transfer failed and had to restart. [Insert graphs: completion time vs RTT, completion time vs loss, winner heatmap]

e. Critical Evaluation / Optimization

Results at low RTT varied by up to 15 % between runs, so 20 repetitions and medians were used. QUIC used more CPU than TCP because it runs in user space, which explains its small loss on the high-bandwidth 5G profile. Disabling NIC offloads and pinning processes to fixed CPU cores reduced variance.

10. Results and Discussion

a. Findings and Inferences

- HTTP/3 was faster in most conditions, and the gap grew with RTT (about 22 % faster at 300 ms).
- Under packet loss of 2 % and above, HTTP/3 completed 25–30 % faster because there is no head-of-line blocking.
- Crossover point: below ~30 ms RTT with under 0.5 % loss and high bandwidth, HTTP/2 matched or slightly beat HTTP/3.
- On the India 4G profile, HTTP/3 was about 23 % faster.
- Only HTTP/3 survived the simulated network switch.

b. Innovation, Novelty, and Evidence

Novelty: emulation of Indian 4G/5G mobile network profiles with variable latency and bursty loss, and locating the exact crossover point between QUIC and TCP.

11. Conclusion

The project built an automated testbed comparing HTTP/2 over TCP and HTTP/3 over QUIC across 50+ network conditions. QUIC performs better on high-latency, lossy and changing networks, which describe typical Indian mobile conditions. TCP remains competitive only on fast, clean, low-latency links. The crossover analysis gives a practical guide for when to enable HTTP/3.

12. Limitations and Future Enhancement

- Emulated network may not fully match real mobile networks.
- Single client/server pair; no multiple concurrent flows.
- Future: real-device tests on Indian carriers, BBR vs CUBIC comparison, video streaming workloads.

References

- RFC 9000 – QUIC: A UDP-Based Multiplexed and Secure Transport.
- RFC 9114 – HTTP/3.
- Nginx HTTP/3 documentation – nginx.org.
- aioquic – github.com/aiortc/aioquic.
- tc-netem(8) Linux manual page.